Recurrent Neural Networks

John Snyder

STAT 9340: Spring 2023

Section 1

RNNs

Recurrent Neural Networks

The initial DNN architecture we learned does not assume any structure which exploits intricacies of any particular data type.

- CNNs use convolution to exploit spatial dependence and find patterns in images.

As we will see, recurrent NNs are built to exploit *temporal* patterns in the input data. Like CNNs shared parameters across space, RNNs will share parameters across time.

Note, both of these technologies have been surpassed in performance by transformer based approaches which we will study later in the course, but are still worth considering.

Recurrence is not a new concept

Recurrent Neural Networks (RNNs) were originally developed in the 1980s to process sequence data. In recent years, they have been very useful especially for language processing applications. These models are closely related to multivariate state-space models for dynamical systems, as one might see in time series, econometrics, or spatio-temporal statistics.

Consider a classical dynamical system:

$$\mathbf{s}_t = f\left(\mathbf{s}_{t-1}; \boldsymbol{\theta}\right)$$

where $\mathbf{s}_t$ represents the state of the system at time t. This is considered "recurrent" because the state at time t refers back to the state at time $t-1$. We can rewrite this in a so-called "unfolded" form:

$$\mathbf{s}_t = f\left(\mathbf{s}_{t-1}; \boldsymbol{\theta}\right) = f\left(f\left(\mathbf{s}_{t-2}; \boldsymbol{\theta}\right); \boldsymbol{\theta}\right) = f\left(f\left(f\left(\mathbf{s}_{t-3}; \boldsymbol{\theta}\right); \boldsymbol{\theta}\right); \boldsymbol{\theta}\right) \ldots$$

Note, the parameters $\boldsymbol{\theta}$ are shared across all states.

The statistical approach

In a traditional dynamical system, we would also have some output, say $\mathbf{y}_t$, that depends on the hidden state, e.g.,

$$\mathbf{y}_t = g\left(\mathbf{s}_t; \theta_g\right).$$

As we have seen, in Statistics we consider both the state evolution and output observation model contain random error terms. We typically assume conditional independence such that:

$$\left[\mathbf{y}_T, \ldots, \mathbf{y}_1 \mid \mathbf{s}_1, \ldots, \mathbf{s}_T, \theta_g\right] = \prod_{t=1}^{T} \left[\mathbf{y}_t \mid \mathbf{s}_t, \theta_g\right]$$

and

$$\left[\mathbf{s}_T, \mathbf{s}_{T-1}, \ldots, \mathbf{s}_0 \mid \theta\right] = \prod_{t=1}^{T} \left[\mathbf{s}_t \mid \mathbf{s}_{t-1}, \theta\right].$$

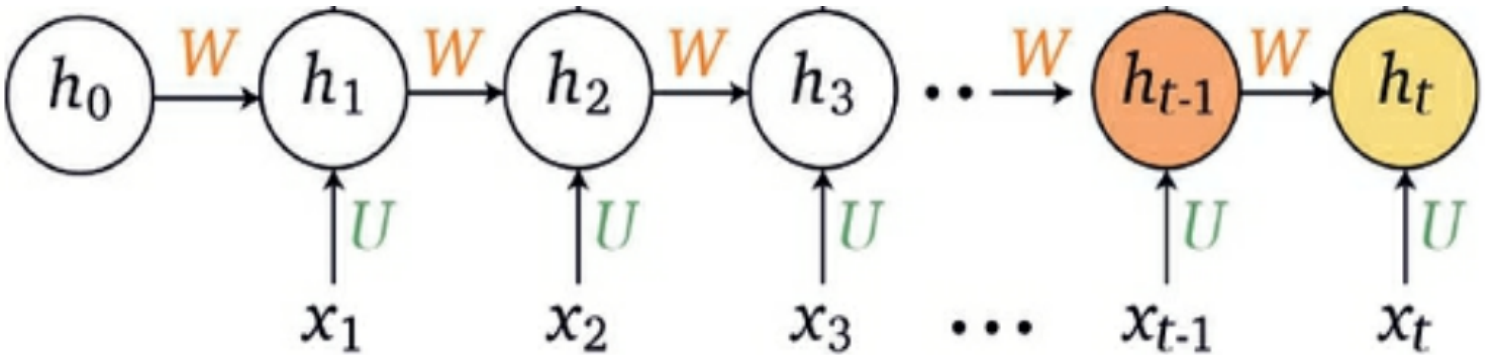
where $\left[\mathbf{y}_t \mid \mathbf{s}_t, \theta_g\right]$ represents a data distribution and $\left[\mathbf{s}_t \mid \mathbf{s}_{t-1}, \theta\right]$ the state evoluation model.

Moving to RNNs

In a RNN, as with a traditional state space model, the states are typically considered "hidden" (we denote the hidden states $\mathbf{h}_t$) and we also may have external inputs $\mathbf{x}_t$:

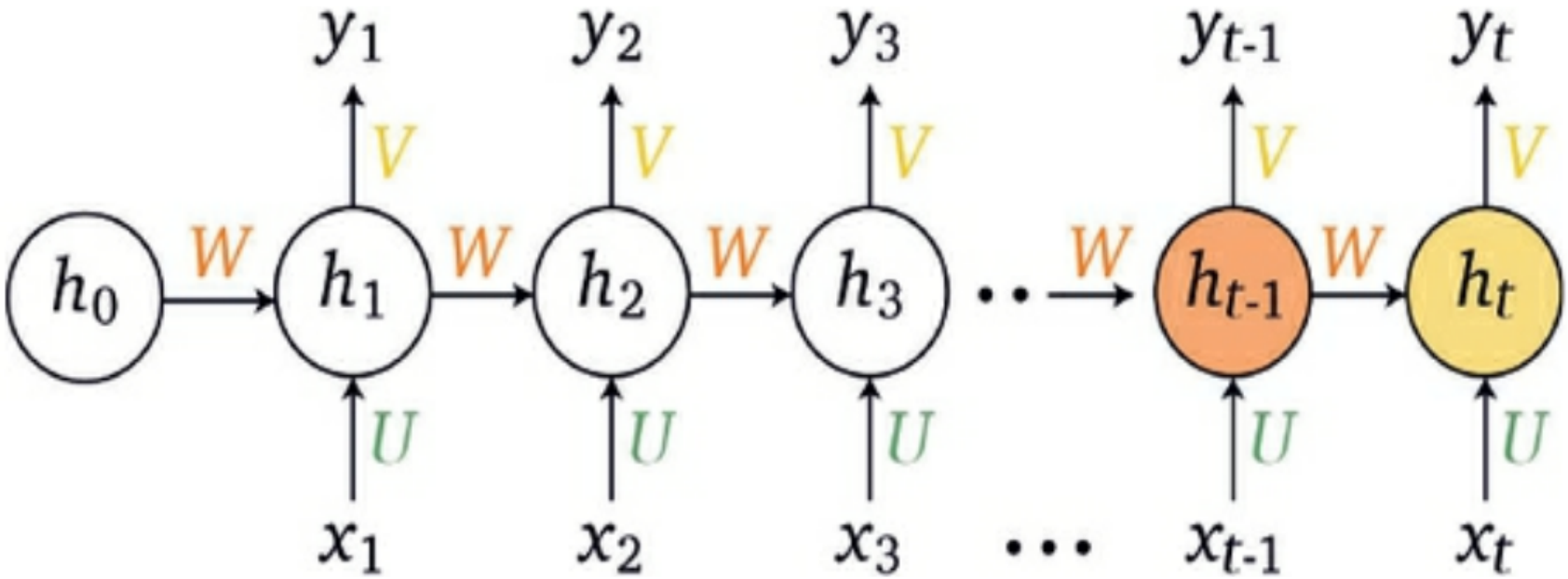
$$\mathbf{h}_t = f\left(\mathbf{h}_{t-1}, \mathbf{x}_t; \theta\right).$$

The following image shows how this is unfolded into a computational graph structure:



Moving to RNNs: Output variables

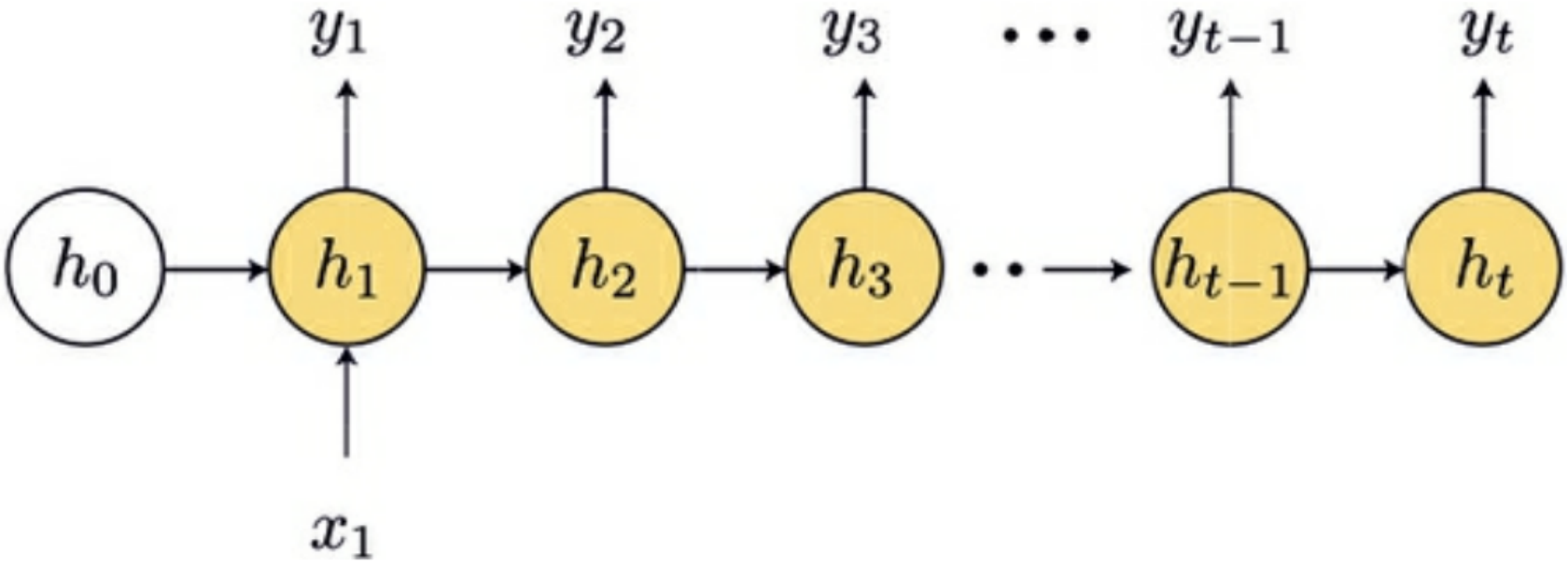
We will have another set of shared weights for moving from the hidden layer to the output.



The length of these input and output sequences is variable!

Flavors of RNN: One to Many

A one to many architecture inputs one value and outputs a sequence.

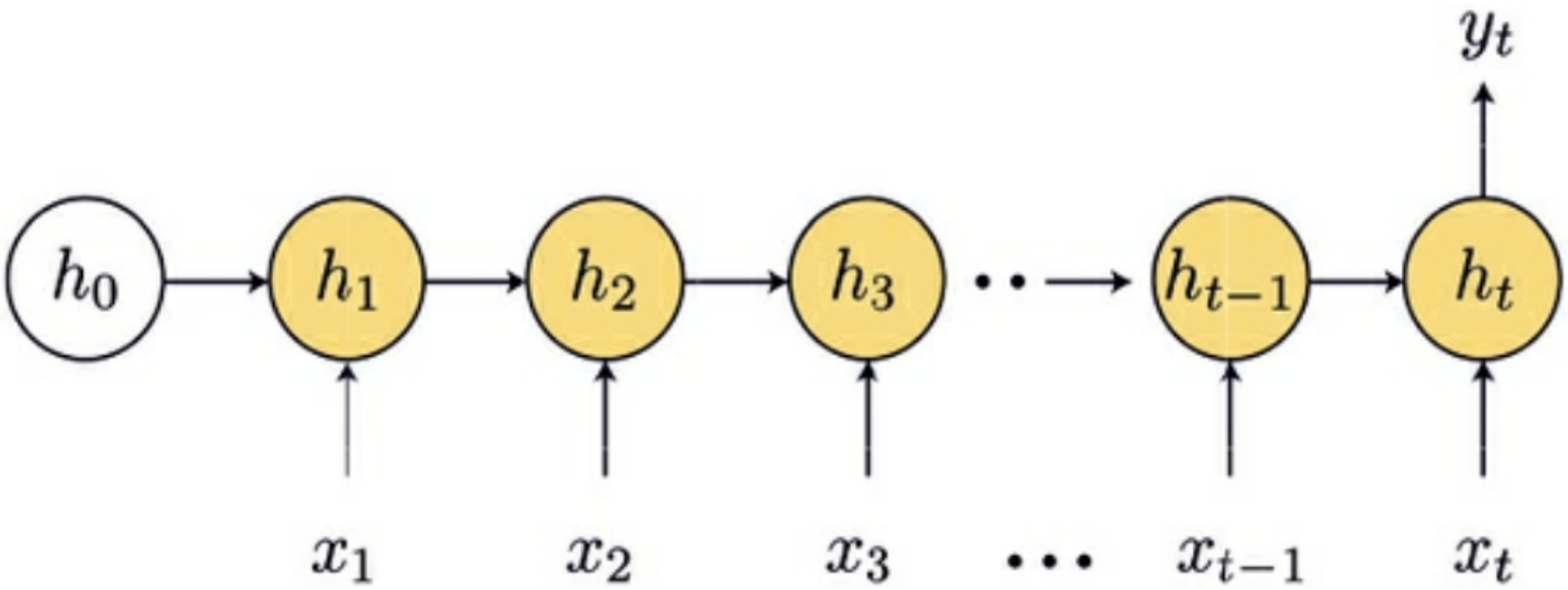


Example use cases:

- Image Captioning

Flavors of RNN: Many to One

A Many to one architecture inputs a sequence and outputs a single value

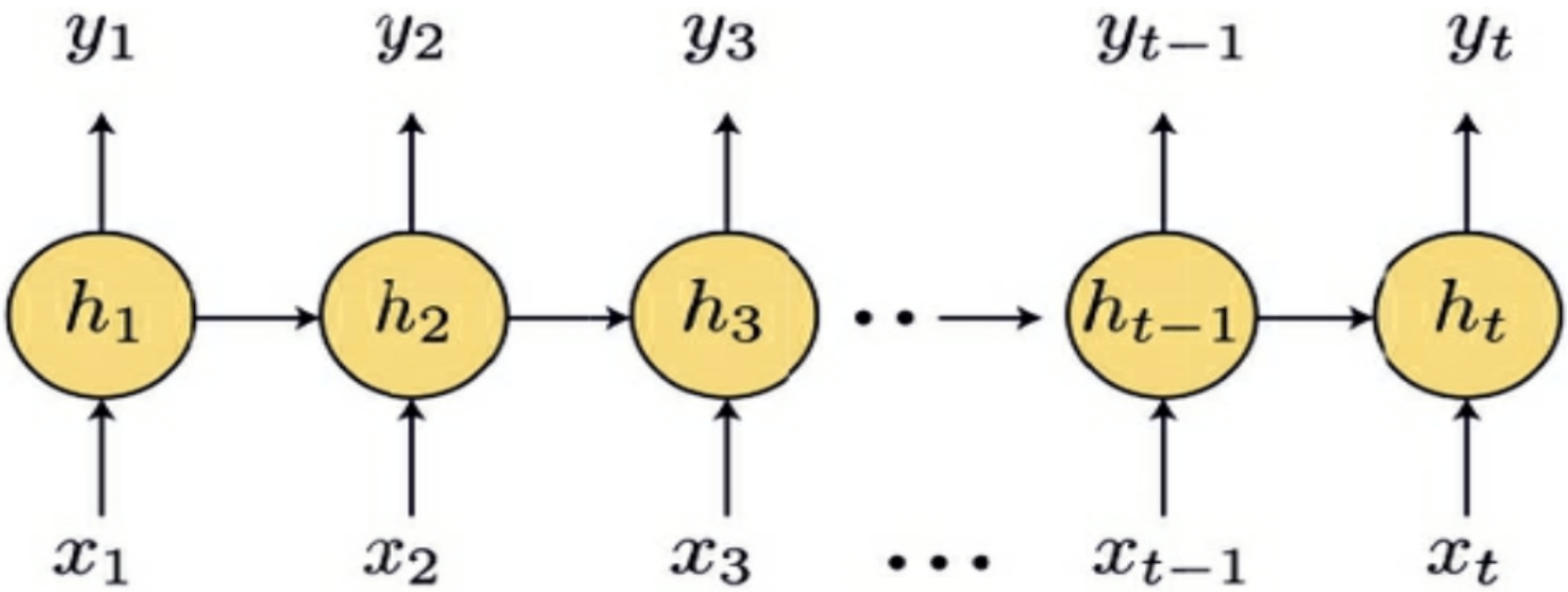


Example use cases:

- Sentiment analysis, Stock price prediction

Flavors of RNN: Many to Many

In a Many to Many architecture, we input and output a sequence

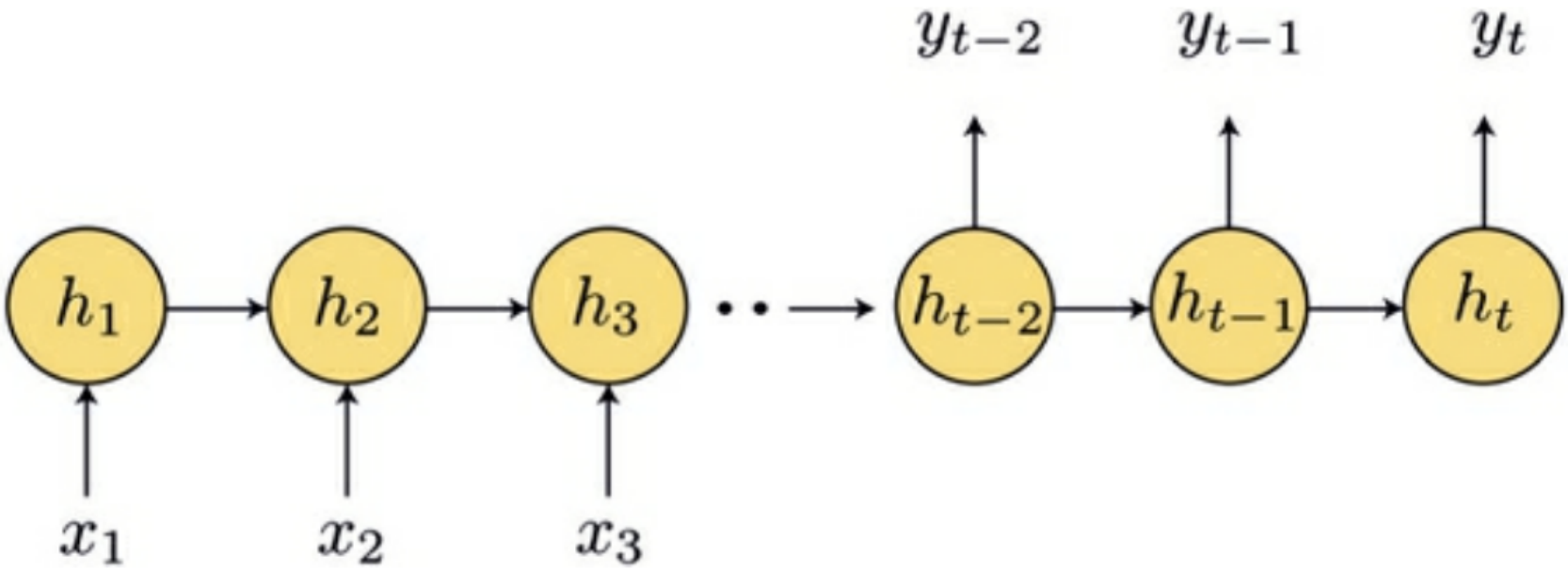


Example use cases:

- time series forecasting, named entity recognition

Flavors of RNN: M2M w/ encoder-decoder

Another flavor of many to many is utilizing an encoder-decoder architecture.



Here, the encoder RNN processes the input sequence and produces a fixed-length context vector, which represents the entire input sequence. The decoder RNN then uses this context vector to generate the output sequence one element at a time.

Example use cases:

- Machine Translation

RNNs

Language Modeling Example (WildML Tutorial, Part 2) Consider a sentence with m words. From a probabilistic perspective, one seeks a language model to predict the probability of observing a sentence in a given data set as:

$$p\left(w_1, w_2, \ldots, w_m\right) = \prod_{i=1}^{m} p\left(w_i \mid w_1, \ldots, w_{i-1}\right).$$

That is, the probability is based on the sequence of probabilities based on the words that come before a given word; e.g., the probability of "He went to buy some chocolate" is the probability of "chocolate" given "He went to buy some", multiplied by the probability of "some" given "He went to buy", etc.

RNN Language Example

Why would we want such a model? One could use it as a way to score potential candidate sentences that emerge from a machine translation algorithm or speech recognition system.

You could also use this as a generative model - given an existing sequence of words, we could sample the next word from the predicted probabilities, and repeat until we get a full sentence. E.g., tutorial Part 4 gives several examples:

- "I am a bot, and this action was performed automatically."
- "There is no problem here, but at least still wave!"
- "It depends on how plausible my judgement is."

RNN

How does one actually take sentences and use them in an RNN model? As we talked about in Data III, there are some preprocessing things that we can do.

- Tokenize Text
- Remove Infrequent Words
- Prepend Special Start and End Tokens
- Encode the words by mapping to an index code and create input and output vectors

Tokenize and Encode example

```
import nltk
nltk.download('punkt') # Download the necessary resources (only required once)
```

```
## True
##
## [nltk_data] Downloading package punkt to /Users/john/nltk_data...
## [nltk_data]   Package punkt is already up-to-date!
```

```
sentence = "What aren't you understanding about this?!" # Typical Reddit comment
tokens = nltk.word_tokenize(sentence) # Tokenize the sentence

# Define a vocabulary from the tokens
vocab = sorted(set(tokens))
word_to_index = {word: i for i, word in enumerate(vocab)}

# Encode the tokens as integers
encoded_tokens = [word_to_index[word] for word in tokens]
# Print the tokens and encoded tokens
print(tokens)
```

```
## ['What', 'are', "n't", 'you', 'understanding', 'about', 'this', '?', '!']
```

```
print(encoded_tokens)
```

```
## [2, 4, 5, 8, 7, 3, 6, 1, 0]
```

RNN Formulation

A basic RNN for can be written

$$\mathbf{y}_t = f(\mathbf{v}_t)$$
$$\mathbf{v}_t = \mathbf{W}_{hy}\mathbf{h}_t$$
$$\mathbf{h}_t = g(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t)$$

for activation functions $f(\cdot), g(\cdot)$, and weight matrices $\mathbf{W}_{hy}, \mathbf{W}_{hh}$, and $\mathbf{W}_{xh}$ are shared across all time steps.

As an example, in a natural language processing (NLP) application, $\mathbf{x}_t$ might be of dimension 800×1, corresponding to the vocabulary size, $\mathbf{h}_t$ might be of dimension, say 100×1, representing "memory" (the larger, the more complex relationships can be accommodated), so $\mathbf{W}_{hy}$ is 800×100, $\mathbf{W}_{hh}$ is 100×100, and $\mathbf{W}_{xh}$ is 100×800, and $\mathbf{v}_t$ and $\hat{\mathbf{y}}_t$ are both of dimension 800×1. There are LOTs of parameters, but importantly, they are the same for all times, t.

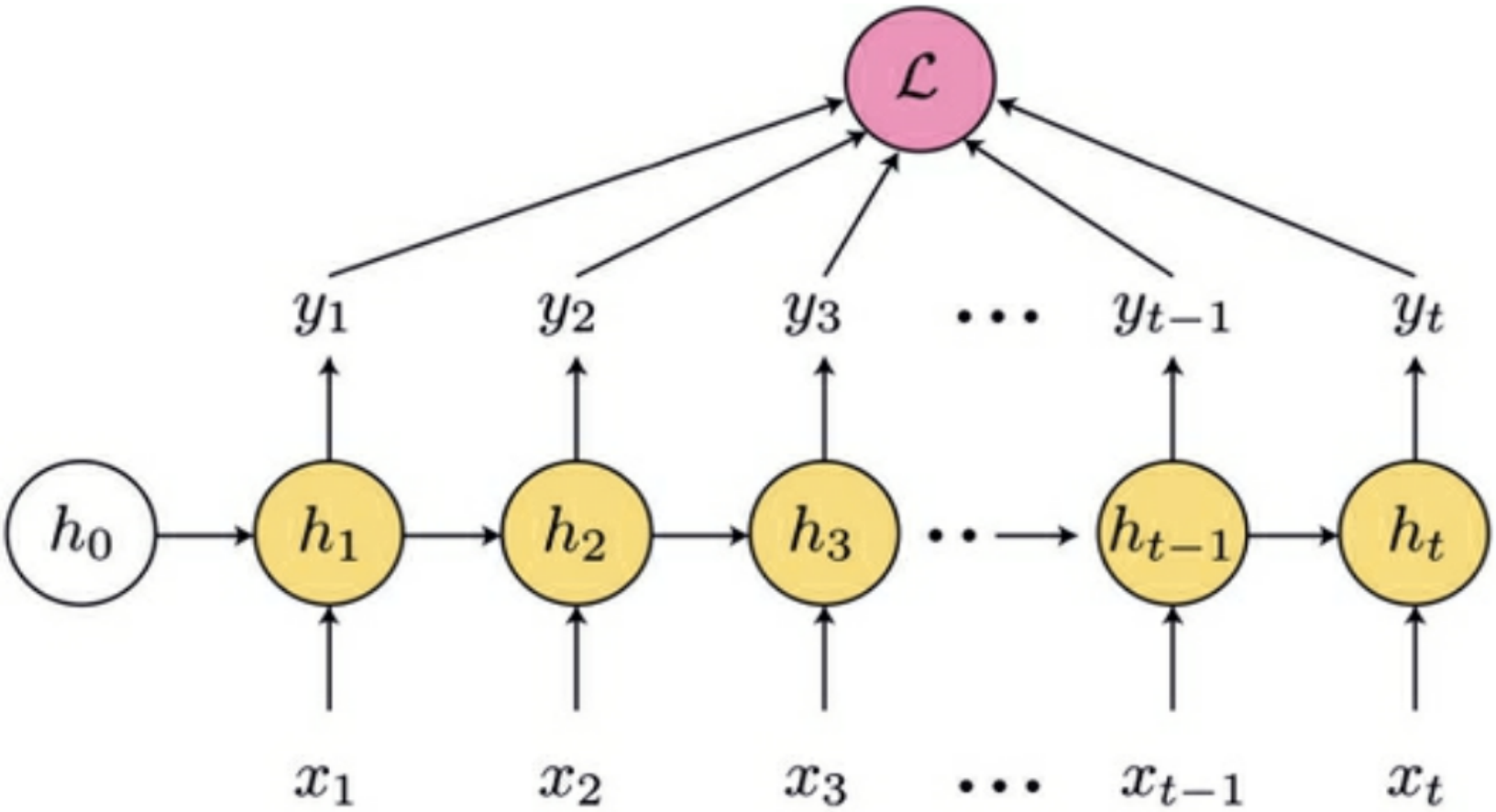
RNN Backpropagation

To estimate the parameters, one would typically define their loss function (e.g., from the log-likelihood) and optimize by back-propagation. However, there is a complication in the case of RNNs because the parameters are common across time. This is traditionally resolved by the back-propagation through time (BPTT) algorithm. Say we have a sequence of length T (i.e., $t = 1, \ldots, T$). Denote $h_t(j)$ as the j th hidden unit where $j = 1, \ldots, N$, $w_{hy}(i, j)$ the weight connecting the j th hidden unit to the i th output unit for $i = 1, \ldots, L$ and $j = 1, \ldots, N$. First, define an objective function. For example,

$$Q = \frac{1}{2} \sum_{t=1}^{T} \|\tilde{\mathbf{y}}_t - \mathbf{y}_t\|^2 = \frac{1}{2} \sum_{t=1}^{T} \sum_{j=1}^{L} (\tilde{y}_t(j) - y_t(j))^2 ,$$

where $\tilde{\mathbf{y}}_t$ represents the observed target output and $\mathbf{y}_t$ is the model output.

RNN Backpropagation



RNN Backpropagation

As usual, we consider a gradient descent algorithm to update the weights:

$$w^{\text{new}} = w - \eta \frac{\partial Q}{\partial w}$$

where η is the learning rate. For notational convenience, define the error terms:

$$\delta_t^y(j) = -\frac{\partial Q}{\partial v_t(j)}, \quad \delta_t^h(j) = -\frac{\partial Q}{\partial u_t(j)}$$

where $\mathbf{v}_t \equiv \mathbf{W}_{hy}\mathbf{h}_t$ and $\mathbf{u}_t \equiv \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t$. These error terms can be computed recursively (see Yu and Deng, 2015, Chp. 13). It is important to note that the same weights apply to all time steps, so this dependence must be factored in.

RNN Backpropagation

Recursive Computation of Error Terms: First, consider the case where $t = T$ (the last time).

$$\delta_T^y(j) = -\frac{\partial Q}{\partial y_T(j)} \frac{\partial y_T(j)}{\partial v_T(j)} = (\tilde{y}_T(j) - y_T(j))\, g'(v_T(j)), \qquad \text{for } j = 1, 2, \ldots, L$$

In vector terms:

$$\delta_T^y = (\tilde{\mathbf{y}}_T - \mathbf{y}_T) \circ g'(\mathbf{v}_T),$$

where $\circ$ is the element-wise product. Similarly, for the hidden layer:

$$\delta_T^h(j) = -\left(\sum_{i=1}^{L} \frac{\partial Q}{\partial v_T(i)} \frac{\partial v_T(i)}{\partial h_T(j)} \frac{\partial h_T(j)}{\partial u_T(j)}\right) = \sum_{i=1}^{L} \delta_T^y(i) w_{hy}(i,j) f'(u_T(j)),$$

for $j = 1, \ldots, N$, or

$$\delta_T^h = \mathbf{W}_{hy}' \boldsymbol{\delta}_T^y \circ f'(\mathbf{u}_T).$$

RNN Backpropagation

Recursive Computation of Error Terms: For $t = T-1, T-2, \ldots, 1$ In vector terms:

$$\boldsymbol{\delta}_t^y = (\tilde{\mathbf{y}}_t - \mathbf{y}_t) \circ g'(\mathbf{v}_t)$$

$$\delta_t^h = \left[\mathbf{W}'_{hh}\delta_{t+1}^h + \mathbf{W}'_{hy}\boldsymbol{\delta}_t^y\right] \circ f'(\mathbf{u}_t)$$

These are calculated recursively, where the error term δ_t^y is propagated back from the output layer at time t and δ_{t+1}^h is propagated back from the hidden layer at time $t+1$.

RNN Backpropagation

Update of the RNN weights:

$$w_{hy}^{\text{new}}(i,j) = w_{hy}(i,j) + \eta \sum_{t=1}^{T} \frac{\partial Q}{\partial v_t(i)} \frac{\partial v_t(i)}{\partial w_{hy}(i,j)} = w_{hy}(i,j) + \eta \sum_{t=1}^{T} \delta_t^y(i) h_t(j)$$

or

$$\mathbf{W}_{hy}^{\text{new}} = \mathbf{W}_{hy} + \eta \sum_{t=1}^{T} \boldsymbol{\delta}_t^y \mathbf{h}_t'$$

Similarly,

$$\mathbf{W}_{xh}^{\text{new}} = \mathbf{W}_{xh} + \eta \sum_{t=1}^{T} \boldsymbol{\delta}_t^h \mathbf{x}_t',$$

$$\mathbf{W}_{hh}^{\text{new}} = \mathbf{W}_{hh} + \eta \sum_{t=1}^{T} \boldsymbol{\delta}_t^h \mathbf{h}_{t-1}'.$$

RNN BPTT

Init: Weights $\mathbf{W}_{hh}, \mathbf{W}_{hy}, \mathbf{W}_{xh}$; Initial hidden units $\mathbf{h}_0$; learning rate η

while *Stopping criterion not met* **do**

  Sample a minibatch of m examples from the training set $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_m\}$ with corresponding targets $\{\boldsymbol{y}_1, \ldots, \boldsymbol{y}_m\}$;

  for $t \leftarrow 1$ **to** T **do**

    $\mathbf{u}_t = \mathbf{W}_{hh}\mathbf{h}_{t-1}\mathbf{W}_{xh}\mathbf{x}_t$;

    $\mathbf{h}_t = f(\mathbf{u}_t)$;

    $\mathbf{v}_t = \mathbf{W}_{hy}\mathbf{h}_t$;

    $\mathbf{y}_t = g(\mathbf{v}_t)$;

  end

  $\delta_T^y = (\tilde{\mathbf{y}}_T - \mathbf{y}_T) \circ g'(\mathbf{v}_T)$;

  $\delta_T^h = \mathbf{W}'_{hy}\delta_T^y \circ f'(\mathbf{u}_T)$;

  for $T \leftarrow T-1, T-2$ **to** 1 **do**

    $\delta_t^y = (\tilde{\mathbf{y}}_t - \mathbf{y}_t) \circ g'(\mathbf{v}_t)$;

    $\delta_t^h = \left[\mathbf{W}'_{hh}\delta_{t+1}^h + \mathbf{W}'_{hy}\delta_t^y\right] \circ f'(\mathbf{u}_t)$;

  end

  $\mathbf{W}_{hy}^{\text{new}} = \mathbf{W}_{hy} + \eta \sum_{t=1}^{T} \delta_t^y \mathbf{h}'_t$;

  $\mathbf{W}_{xh}^{\text{new}} = \mathbf{W}_{xh} + \eta \sum_{t=1}^{T} \delta_t^h \mathbf{x}'_t$;

  $\mathbf{W}_{hh}^{\text{new}} = \mathbf{W}_{hh} + \eta \sum_{t=1}^{T} \delta_t^h \mathbf{h}'_{t-1}$;

end

RNN Vanishing Gradients

The biggest challenge to implementing the BPTT optimization is the so-called vanishing gradient/exploding gradient problem.

The issue is that when the gradients get propagated over many times they tend to either vanish (most of the time) or explode (this doesn't happen as often). This can be seen from the BPTT algorithm as shown above.

Although there are ways in which one can mitigate exploding gradient issue by ensuring stability (see Yu and Deng, 2015) the series of dependencies through time will tend to lead to exponentially smaller weights. This can also be mitigated in some cases, but the most popular approach is to consider a different type of RNN that is less sensitive to this problem. Such models add "gates" that selectively allow information from the past to influence the presence.

The long short-term memory architecture

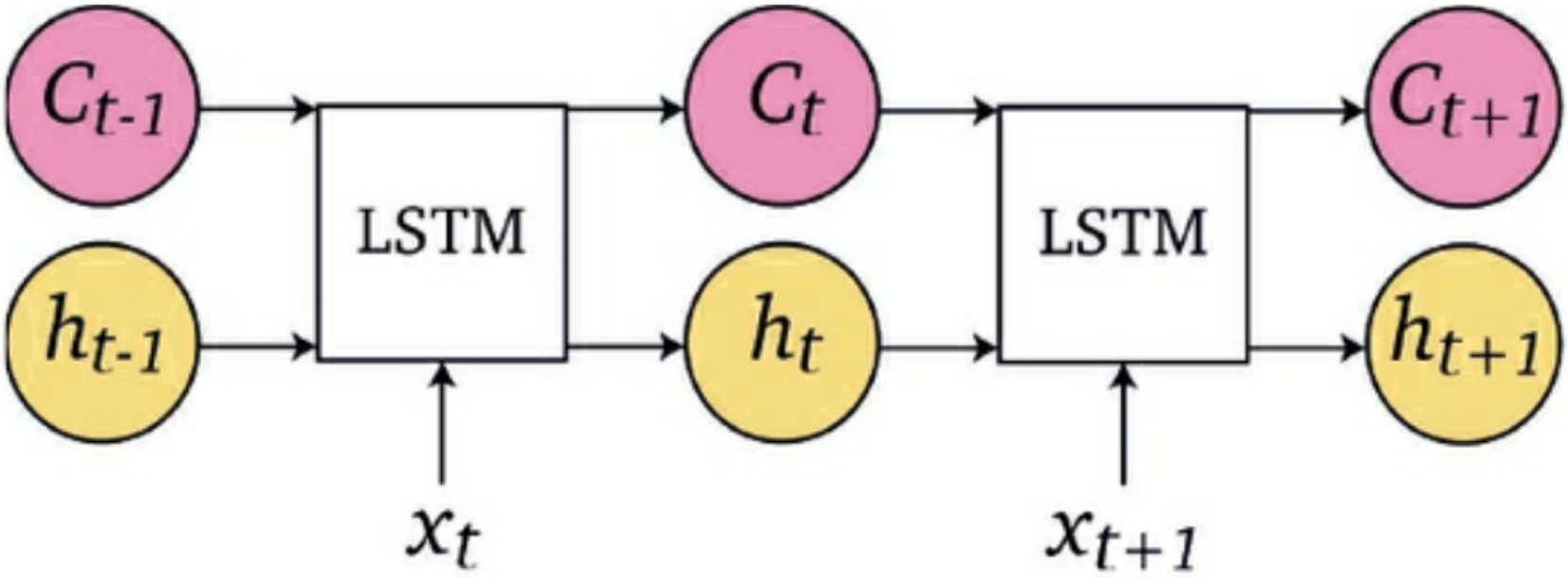
Historically, the most effective gated RNN has been the long short-term memory (LSTM) RNN. The idea is to create time paths that have gradients that to not vanish or explode.

At each time step t the LSTM receives as input the current state x_t, the hidden state h_{t-1}, and *memory cell* c_{t-1} of the previous time step, and outputs the hidden state h_t and memory cell c_t at time t. The memory cells propagate information from the previous state to the next, whereas the hidden states determine the way in which that information is propagated.

Consider:

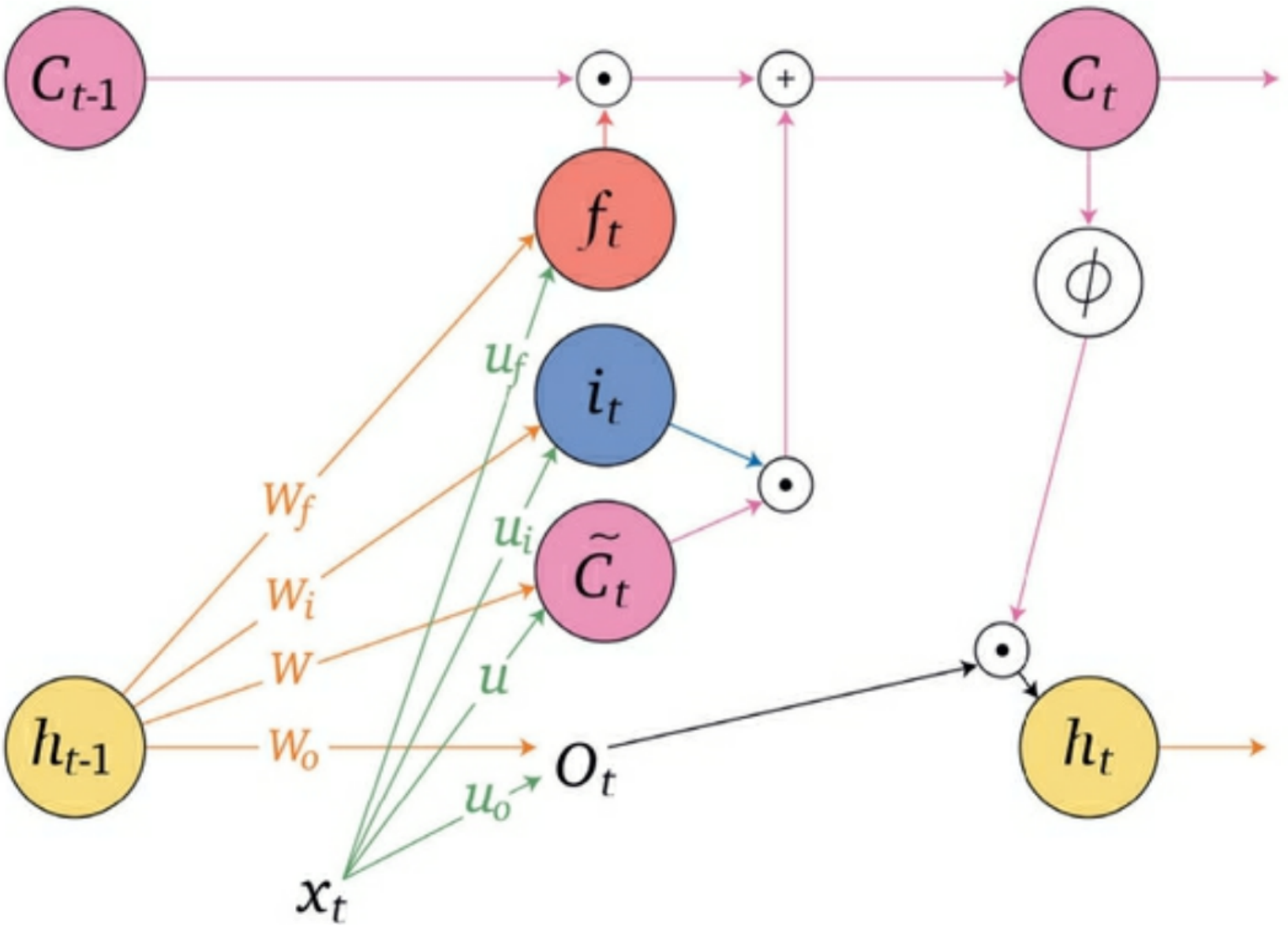
$$
\begin{aligned}
f_t &= \sigma\left(W_f h_{t-1} + U_f x_t\right) && \text{forget gate} \\
i_t &= \sigma\left(W_i h_{t-1} + U_i x_t\right) && \text{input gate} \\
o_t &= \sigma\left(W_o h_{t-1} + U_o x_t\right) && \text{output gate} \\
\tilde{c}_t &= \phi\left(W h_{t-1} + U x_t\right) && \text{candidate memory} \\
c_t &= f_t c_{t-1} + i_t \tilde{c}_t && \text{memory cell} \\
h_t &= o_t \phi\left(c_t\right) && \text{output gated memory}
\end{aligned}
$$

LSTM



The memory cells allow information to propagate forward through the network without directly being activated, which helps avoid vanishing gradients.

LSTM

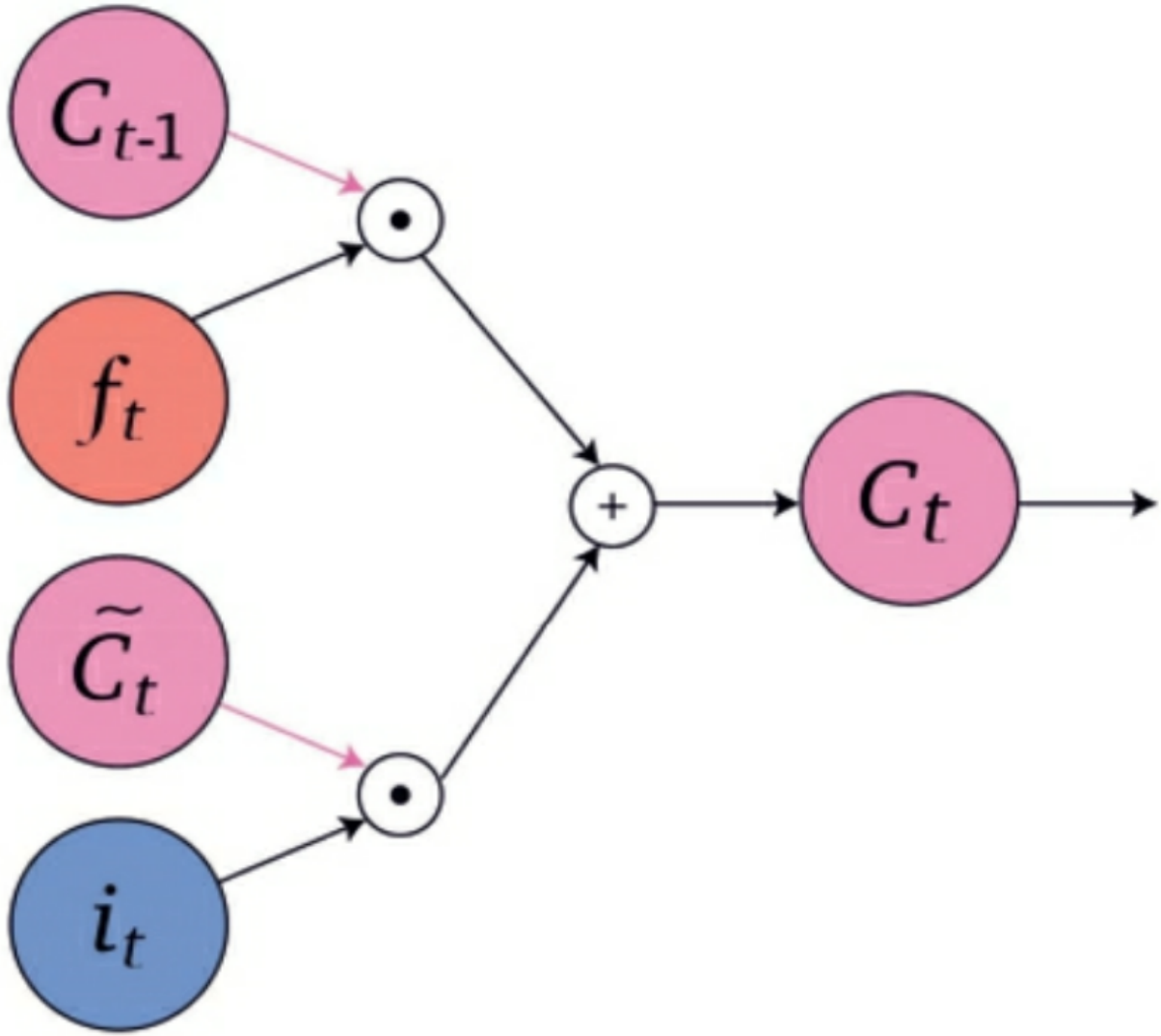


LSTM component notes

- The forget gate f_t is a non-linear function, a sigmoid, of a combination of the current state x_t and the previous hidden state h_{t-1}
- If $i_t = 0$ then the new candidate memory $\tilde{c}_t$ is ignored
- The memory c_t is updated by partially forgetting the previous memory c_{t-1} and adding the new candidate memory $\tilde{c}_t$
- The new candidate memory c_t is a non-linear function, a sigmoid, of a combination of the current state x_t and the previous hidden state h_{t-1}
- The output gate O_t is a non-linear function, a sigmoid, of a combination of the current state x_t and the previous hidden state h_{t-1}

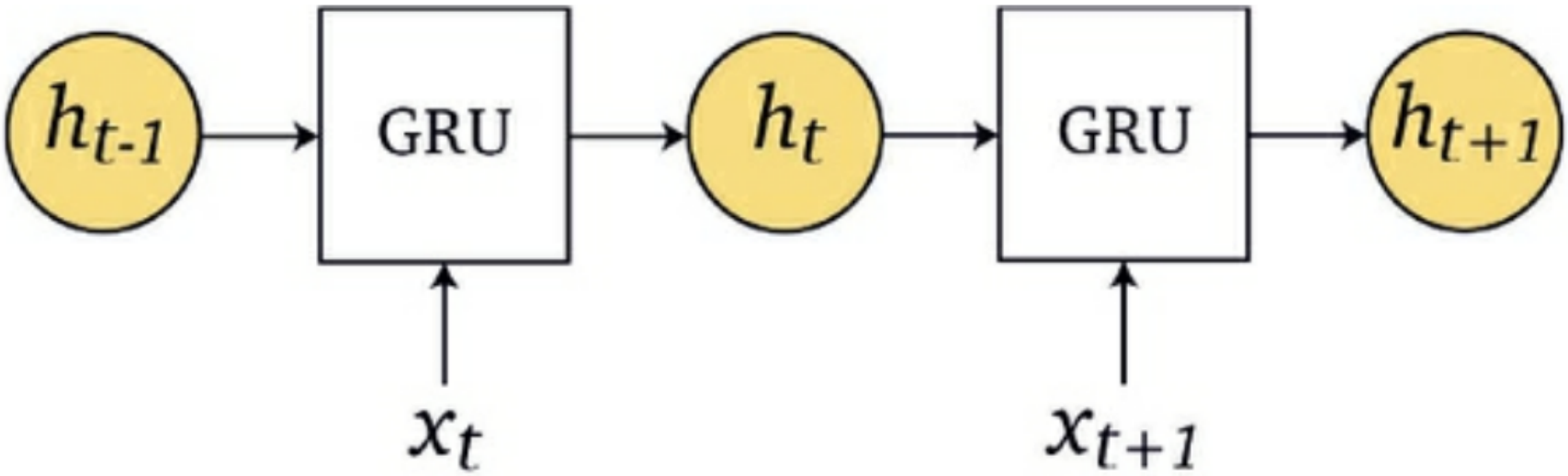
LSTM

$$c_t = f_t c_{t-1} + i_i \tilde{c}_t$$



Gated Recurrent Units (GRUs)

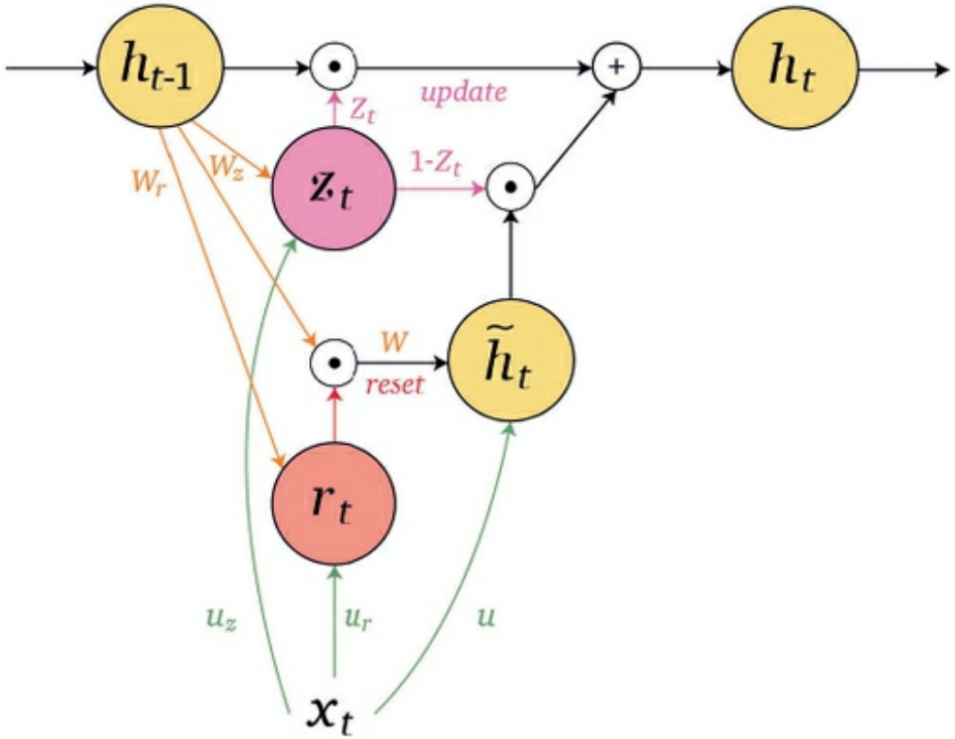
GRUs have a similar overall architecture to what we have seen already



GRUs (cont)

$$z_t = \sigma\left(W_z h_{t-1} + U_z x_t\right)$$
$$r_t = \sigma\left(W_r h_{t-1} + U_r x_t\right)$$
$$\tilde{h}_t = \phi\left(W\left(r_t \odot h_{t-1}\right) + U x_t\right)$$
$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$

- z_t is the *update gate*
- r_t is the *reset gate*
- $\tilde{h}_t$ is the *candidate*
- h_t is the *output*



GRU notes

- The activation h_t at time t is a linear interpolation between the previous activation h_{t-1} and the current candidate $\tilde{h}_t$, which is controlled by an update gate z_t, which itself is a combination of the current state x_t and the previous hidden state h_{t-1}.
  - If the update gate is set to $z_t = 0$, then the output is the new candidate
  - If the update gate is set to $z_t = 1$, then the output is the previous hidden state, ignoring both the candidate and current state altogether
- The candidate activation $\tilde{h}_t$ is a *tanh* activation of a combination of the current state x_t and the previous hidden state h_{t-1} modulated by the reset gate r_t.
- The reset gate r_t is a sigmoid activation of a combination of the current state x_t and the previous hidden state h_{t-1}
  - If the reset gate is set to $r_t = 0$ then the candidate $\tilde{h}_t$ is a function of the current state x_t such that $\tilde{h}_t = \phi(Ux_t)$, forgetting the previous hidden state h_{t-1}
  - If the reset gate is set to $r_t = 1$ then the candidate is a function of the previous hidden state h_{t-1}, ignoring the current state x_t

GRU notes(cont)

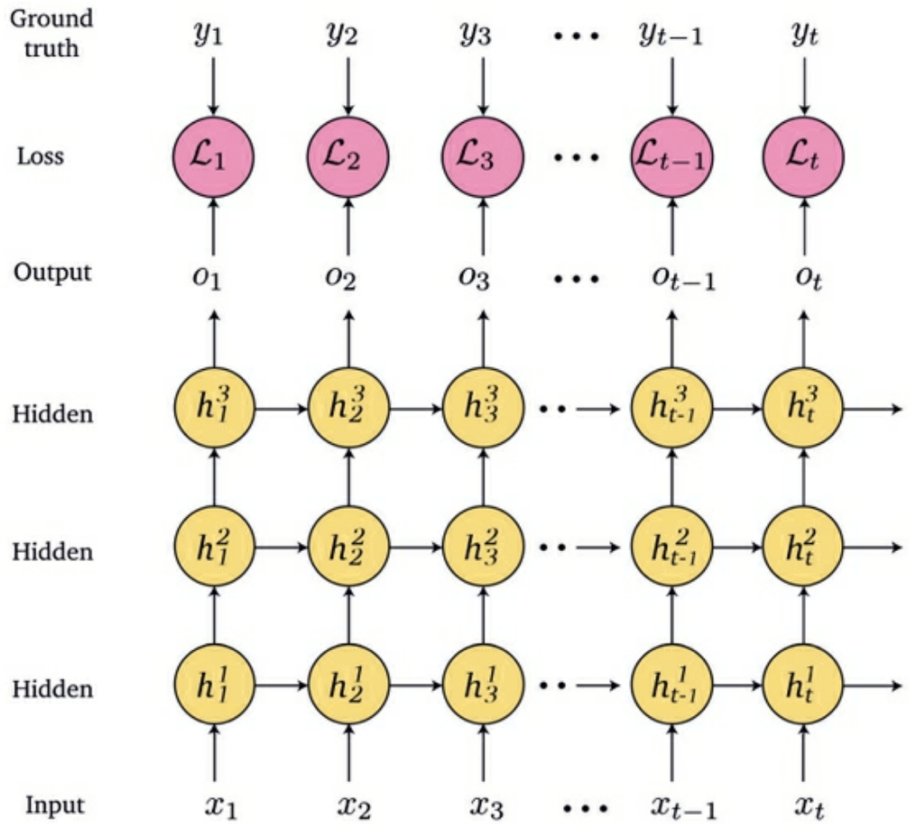
- If $z_t = 0$ and $r_t = 0$ then the output hidden state is only dependent on the current state $\phi(Ux_t)$, forgetting the previous hidden state h_{t-1}
- If $z_t = 0$ and $r_t = 1$, then the GRU is reduced to an RNN

As an analogy, consider the input x_t to be the weather today, the hidden unit h_{t-1} to be the clothes we wore yesterday, $\tilde{h}_t$ to be the candidate clothes we prepared to wear today and h_t to be the actual clothes we wear today. Usually, we wear clothes based on the weather, based on what we wore yesterday, and based on our mood or what we prepared to wear today. The update and reset gates determine to what extent we take into account these factors: Do we ignore the weather x_t completely? Do we forget what we wore yesterday h_{t-1} ? And do we take into account our candidate clothes we prepared $\tilde{h}_t$, and to what extent?

Deep RNNs

We can also have a deep RNN model, where the recurrent hidden layer for a layer l would be structured as

$$\mathbf{h}_t^l = g\left(\mathbf{W}_{h^l h^l}\mathbf{h}_{t-1}^l + \mathbf{W}_{h^{l-1} h^l}\mathbf{h}_{t-1}^{l-1}\right).$$



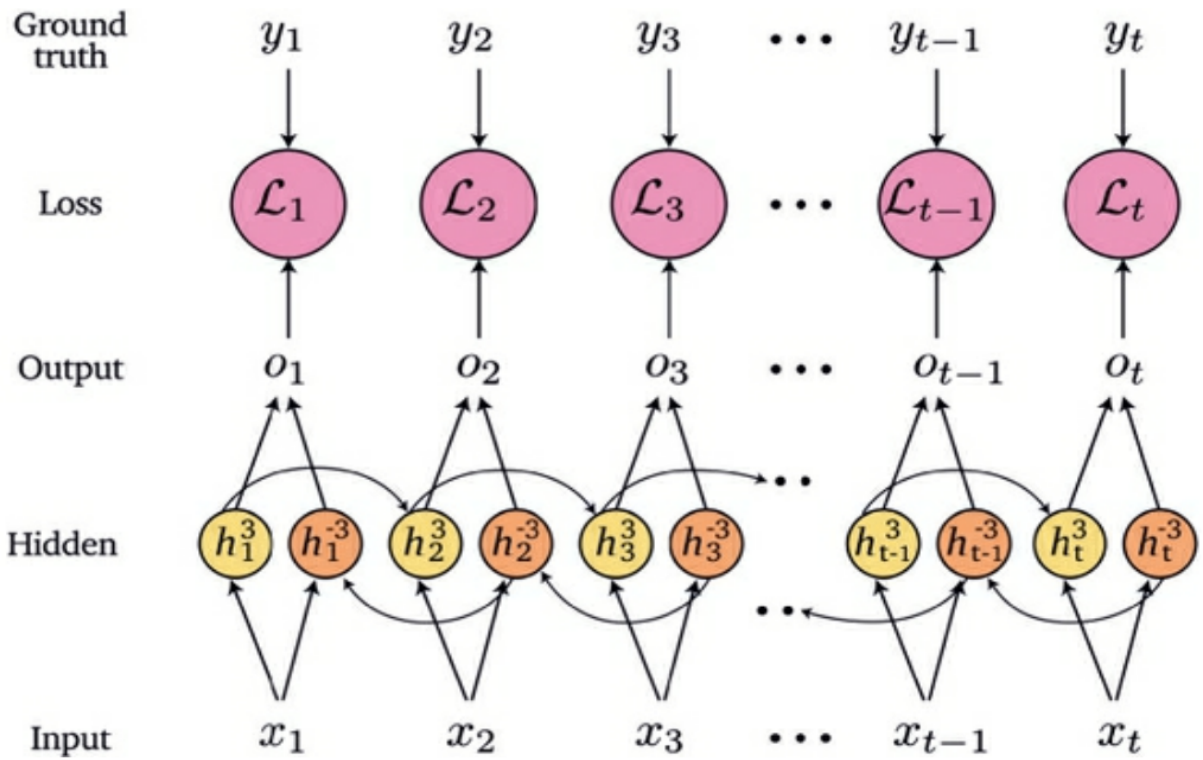
Bidirectional RNNs

Often (language applications) we need to capture dependencies in both directions, *bidirectional* RNNs facilitate this.

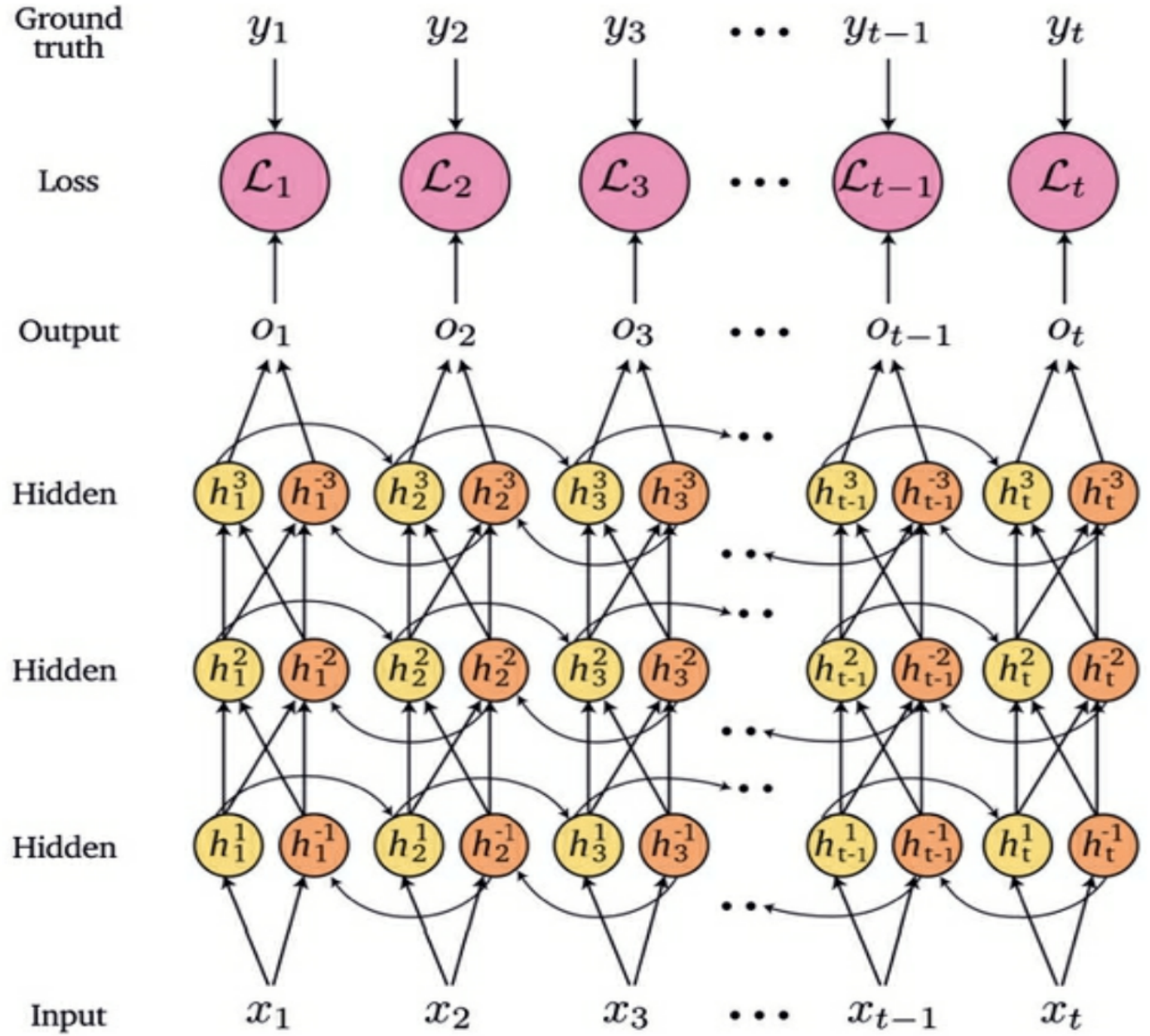
$$\mathbf{h}_t = g\left(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t\right)$$

$$\bar{\mathbf{h}}_t = g\left(\bar{\mathbf{W}}_{hh}\bar{\mathbf{h}}_{t+1} + \mathbf{W}_{xh}\mathbf{x}_t\right)$$

$$\mathbf{v}_t = \mathbf{W}_{hy}\left[\mathbf{h}_t; \bar{\mathbf{h}}_t\right]^T$$



Deep Bidirectional LSTM



Sequence to Sequence models

Building a language model involves a conditional distribution over words in the language and as discussed it is important to generate entire sequences from this.

Sequence-to-sequence (seq2seq) models (Sutskever et al., 2014) consider entire sequences, or sentences, as inputs and outputs. Seq2seq models consist of an encoder and a decoder.

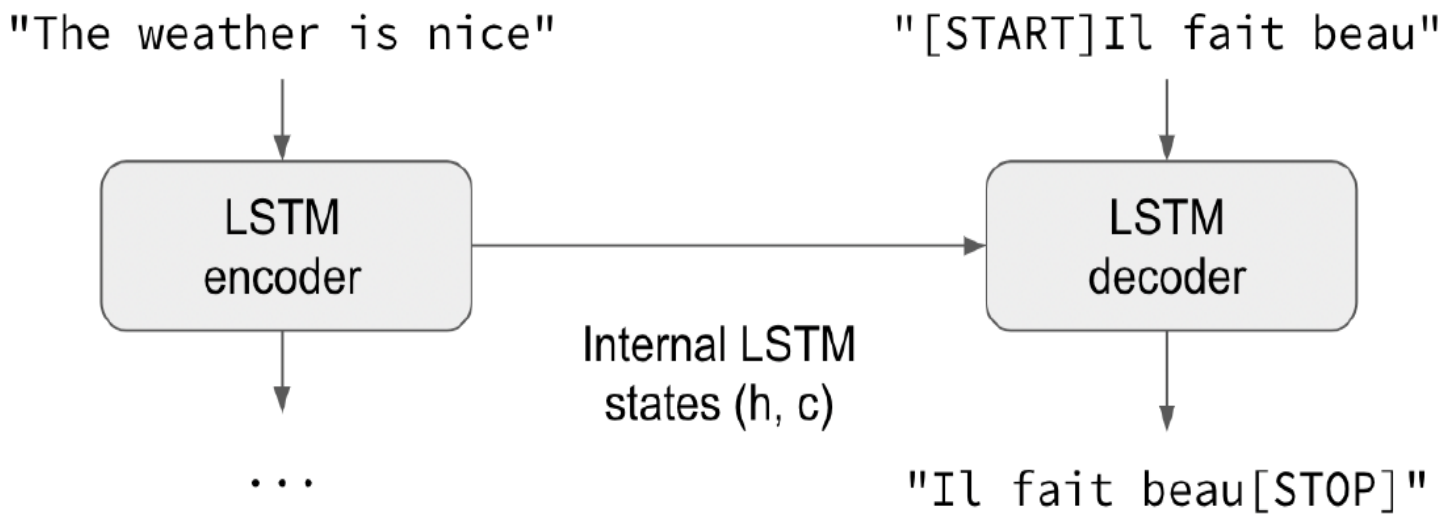
- The encoder is an RNN(GRU/LSTM), which may be bidirectional and/or deep, which encodes the input sequence $(x_1, \ldots, x_s)$ into a context vector as output $z = \text{encoder}\,(x_1, \ldots, x_s)$.
- The decoder is also an RNN(GRU/LSTM) that receives the context vector z as input, as the first hidden state vector, and generates an output sequence $(y_1, \ldots, y_t) = \text{decoder}(z)$. The encoder and decoder models are trained end-to-end such that:

$$(y_1, \ldots, y_t) = \text{decoder}\,(\text{encoder}\,(x_1, \ldots, x_s))$$

seq2seq in language translation

Essentially the network learns

- How to create a latent representation of an input sentence in *any* language
- How to bring that latent representation into any language



seq2seq in language translation

- The general idea of this kind of model is that: we have a set of sequential inputs $\mathbf{x} = (x_1, \ldots, x_{T_x})$ and corresponding sequential outputs $\mathbf{y} = \left(y_1, \ldots, y_{T_y}\right)$. In general, $T_x \neq T_y$.
- From a probabilistic perspective, our goal is to find the maximum conditional density $P(\mathbf{y} \mid \mathbf{x})$.
- We usually model it by encoding the input sequence $\mathbf{x}$ to some latent state by $\mathbf{s} = \text{Encoder}(\mathbf{x})$ (where the encoder is usually some RNN)
- Then given the input sequence, the output is generated through another RNN whose input states are modeled by the hidden states (where f is an RNN):

$$h(t) = f(h(t-1), y(t-1), \mathbf{s})$$

- Then the next output y_t is generated by (where g is typically a softmax):

$$P\left(y_t \mid y_{\leq t-1}\right) = g(h(t), y(t-1), \mathbf{s})$$

Section 2

Implimentations of Select Architectures

Key Tensorflow layers for RNN architectures

As usual, Tensorflow has some built in layers that make training various RNN architectures easy, as well as others to augment the functionality of the below

- `tf.keras.layers.LSTM`: implements the LSTM architecture
  - `units`: The number of LSTM units in the layer.
  - `return_sequences`: Whether to return the full sequence of outputs or only the last output.
  - `dropout`: The fraction of the input units to drop during training.
  - `recurrent_dropout`: The fraction of the recurrent units to drop during training.
  - Some additional options to change gating/flow mechanics
- `tf.keras.layers.SimpleRNN`: implements the LSTM architecture
- `tf.keras.layers.GRU`: implements the LSTM architecture
  - Same as LSTM, with a few extras to provide granular control over gating mechanics

Other important layers for RNNs

- `tf.keras.layers.Embedding`: learn an embedding of the input
  - `input_dim`: The size of the vocabulary.
  - `output_dim`: The size of the dense embedding vectors.
  - `input_length`: The length of the input sequences.
- `tf.keras.layers.Bidirectional`: applies a forward and backward LSTM to the input sequences
  - `layer`: The type of layer to apply in the forward and backward directions (e.g., LSTM, GRU).
- `tf.keras.layers.TimeDistributed`: apply a dense layer to each time step of the output sequence
  - `layer`: The type of layer to apply to each time step of the output sequence.

Simple one layer many to one RNN model

The below is the structure of a basic RNN implementation.

- LSTM would be just use the corresponding layer in place of `SimpleRNN`

```
import tensorflow as tf

input_shape = (None, 10)
inputs = tf.keras.layers.Input(shape=input_shape)
x = tf.keras.layers.SimpleRNN(64)(inputs)
outputs = tf.keras.layers.Dense(1, activation='sigmoid')(x)
simple_model = tf.keras.models.Model(inputs=inputs, outputs=outputs)
```

Summary of Simple Model

```
simple_model.summary()
```

```
## Model: "model"
## _________________________________________________________________
##  Layer (type)                 Output Shape              Param #
## =================================================================
##  input_1 (InputLayer)         [(None, None, 10)]        0
##
##  simple_rnn (SimpleRNN)       (None, 64)                4800
##
##  dense (Dense)                (None, 1)                 65
##
## =================================================================
## Total params: 4,865
## Trainable params: 4,865
## Non-trainable params: 0
## _________________________________________________________________
```

"Deep" many to one RNN model

When using a deep RNN architecture, recall the entire sequence of hidden units must be passed onto the next recurrent layer.

```
import tensorflow as tf

input_shape = (None, 10)
inputs = tf.keras.layers.Input(shape=input_shape)
x = tf.keras.layers.SimpleRNN(128, return_sequences=True)(inputs)
x = tf.keras.layers.SimpleRNN(64, return_sequences=True)(x)
x = tf.keras.layers.SimpleRNN(32)(x)
outputs = tf.keras.layers.Dense(1, activation='sigmoid')(x)
deep_rnn_model = tf.keras.models.Model(inputs=inputs, outputs=outpu
```

Summary of Deep RNN Model

```
deep_rnn_model.summary()
```

```
## Model: "model_1"
## _________________________________________________________________
##  Layer (type)                 Output Shape              Param #
## =================================================================
##  input_2 (InputLayer)         [(None, None, 10)]        0
##
##  simple_rnn_1 (SimpleRNN)     (None, None, 128)         17792
##
##  simple_rnn_2 (SimpleRNN)     (None, None, 64)          12352
##
##  simple_rnn_3 (SimpleRNN)     (None, 32)                3104
##
##  dense_1 (Dense)              (None, 1)                 33
##
## =================================================================
## Total params: 33,281
## Trainable params: 33,281
## Non-trainable params: 0
## _________________________________________________________________
```

"Deep" many to *two* RNN model

```
import numpy as np
import tensorflow as tf

input_shape = (None, 10)

# Define the LSTM layers
lstm_layer1 = tf.keras.layers.LSTM(64, return_sequences=True)
lstm_layer2 = tf.keras.layers.LSTM(32, return_sequences=True)

# Define the input layer
inputs = tf.keras.layers.Input(shape=input_shape)

# Connect the input to the LSTM layers
x = lstm_layer1(inputs)
x = lstm_layer2(x)

# Slice the inputs to two output layers
output1=tf.keras.layers.Dense(1, activation='sigmoid', name='output1')(x[:,-2,:])
output2=tf.keras.layers.Dense(1, activation='sigmoid', name='output2')(x[:,-1,:])

m22_model = tf.keras.models.Model(inputs=inputs, outputs=[output1, output2])
```

Summary of many to *two* RNN model

```
m22_model.summary()
```

```
## Model: "model_2"
## _________________________________________________________________________________________________
##  Layer (type)                   Output Shape         Param #     Connected to
## =================================================================================================
##  input_3 (InputLayer)           [(None, None, 10)]   0           []
##
##  lstm (LSTM)                    (None, None, 64)     19200       ['input_3[0][0]']
##
##  lstm_1 (LSTM)                  (None, None, 32)     12416       ['lstm[0][0]']
##
##  tf.__operators__.getitem (Slic  (None, 32)          0           ['lstm_1[0][0]']
##  ingOpLambda)
##
##  tf.__operators__.getitem_1 (Sl  (None, 32)          0           ['lstm_1[0][0]']
##  icingOpLambda)
##
##  output1 (Dense)                (None, 1)            33          ['tf.__operators__.getitem[0][0]'
##                                                                  ]
##
##  output2 (Dense)                (None, 1)            33          ['tf.__operators__.getitem_1[0][0
##                                                                  ]']
##
## =================================================================================================
## Total params: 31,682
## Trainable params: 31,682
## Non-trainable params: 0
## _________________________________________________________________________________________________
```

Bidirectional RNN

```
import tensorflow as tf
from tensorflow.keras.layers import Input, LSTM, Bidirectional, Dens

input_shape = (None, 10)
inputs = tf.keras.layers.Input(shape=input_shape)
x = Bidirectional(tf.keras.layers.SimpleRNN(64))(inputs)
outputs = tf.keras.layers.Dense(1, activation='sigmoid')(x)
bidirectional_model = tf.keras.models.Model(inputs=inputs, outputs=
```

Summary of Bidirectional RNN

```
bidirectional_model.summary()
```

```
## Model: "model_3"
## _________________________________________________________________
##  Layer (type)                Output Shape              Param #
## =================================================================
##  input_4 (InputLayer)        [(None, None, 10)]        0
##
##  bidirectional (Bidirectiona  (None, 128)              9600
##  l)
##
##  dense_2 (Dense)             (None, 1)                 129
##
## =================================================================
## Total params: 9,729
## Trainable params: 9,729
## Non-trainable params: 0
## _________________________________________________________________
```